

WORKBENCH

Development Roadmap

Working title - React learning project to practical cloud application

Roadmap principle

Build the simplest useful version first. Introduce new React ecosystem tools only when Workbench has developed the problem that the tool solves. Each version should remain usable, tested, and mergeable.

Purpose. Workbench begins as a small React + TypeScript task/project tool for immediate personal use. It can then evolve into a server-backed, cloud-persistent application and potentially a polished portfolio piece. Longer term, it may become the first concrete subsystem of the broader whole-life-planner concept, without forcing that larger scope into the early versions.

Version overview

Version	Primary focus	Key additions	Main learning outcome
0.1	React foundations	Components, props, local state, task UI, tests	Think in React and establish the test-first development rhythm
0.2	Useful local application	Filtering, editing, localStorage, component refinement	Derived state, forms, persistence and cleaner component boundaries
0.3	Shared client state	Zustand and application-state refactor	Experience why a shared state store is useful before adopting one
0.4	Persistent backend	REST API, Zod, database	Separate frontend concerns from API and persistence concerns
0.5	Server-state management	TanStack Query, mutations, invalidation	Treat remote data as server state rather than ordinary React state
0.6	Multi-user cloud data	Authentication, per-user data, cloud database	Identity, authorization and production persistence
1.0	Portfolio-ready product	Cloud deployment, responsive UI, accessibility, documentation	Ship and explain a coherent full-stack product

Workbench 0.1 | React foundations

Build a small but real task list in plain React so the core component/state model becomes instinctive.

Scope

- React + TypeScript + Vite
- Task domain type
- TaskList and TaskItem component composition
- Local state with useState
- Task completion toggling
- Accessible checkbox labels
- Vitest + React Testing Library + user-event + jest-dom
- Dark development theme

Development principles

- Keep dependencies deliberately small.
- Prefer user-facing tests over tests of internal implementation details.
- No backend, global state library, routing framework or UI framework yet.

Exit criteria

- The task list renders from typed data.
- A task can be toggled complete/incomplete through the UI.
- Toggle behaviour is covered by passing tests.
- Changes are developed in focused branches and merged through pull requests.

Current progress note: the initial task-list slice, task toggling, accessibility labelling and first toggle tests have already been implemented and merged. Workbench 0.1 can continue with only the remaining foundation work that proves useful before moving to 0.2.

Workbench 0.2 | Useful local application

Turn the initial React exercise into a tool that is genuinely useful day to day while still remaining entirely local.

Scope

- Refine component structure as genuine pressure appears
- Add task creation and editing
- Add task filtering (for example: all/open/complete)
- Introduce additional task fields only when useful
- Persist state to localStorage
- Expand behaviour-focused test coverage

Development principles

- Learn the difference between stored state and derived state.

- Use ordinary React forms before introducing form libraries.
- Use effects only for genuine side effects such as persistence.
- Let component boundaries emerge from usage rather than premature folder architecture.

Exit criteria

- Workbench survives a browser refresh.
- Tasks can be created/edited and filtered.
- The UI remains understandable and keyboard-usable.
- Tests protect the main local workflows.

Workbench 0.3 | Shared client state with Zustand

Allow Workbench to outgrow prop drilling naturally, then refactor shared client-only state behind a lightweight store.

Scope

- Introduce Zustand
- Move appropriate shared UI/application state into stores
- Use selectors so components subscribe only to the state they need
- Keep domain and presentation concerns separable

Development principles

- Adopt Zustand because a real shared-state problem now exists.
- Compare the before/after architecture rather than treating the store as magic.
- Do not move every value into global state merely because a store exists.

Exit criteria

- The application has clearer state ownership.
- Shared state no longer requires awkward callback/prop chains.
- Local component state remains local where appropriate.
- Existing behaviour remains protected by tests during the refactor.

Workbench 0.4 | REST API and persistent database

Move persistence out of the browser and establish a real full-stack boundary.

Scope

- Create a REST API (Node/Express or similar)
- Define request/response contracts
- Validate runtime data with Zod
- Add persistent database storage
- Replace localStorage as the authoritative persistence layer
- Add API-focused tests

Development principles

- Separate compile-time TypeScript types from runtime validation.
- Design explicit API boundaries.
- Keep the frontend usable while persistence moves server-side.
- Treat database choice as an implementation decision, not a UI concern.

Exit criteria

- Tasks/projects persist independently of one browser.
- Invalid API data is rejected predictably.
- Frontend and backend can be tested separately.
- The API is documented well enough for another client to consume it.

Workbench 0.5 | TanStack Query and server-state management

Replace hand-built remote-fetch lifecycle code with a dedicated server-state abstraction.

Scope

- Introduce TanStack Query
- Query keys for server-state identity
- Caching and freshness/staleness rules
- Mutations for create/update/delete operations
- Targeted query invalidation
- Loading, error and background-refetch UX

Development principles

- Clearly distinguish client state from server state.
- Experience the value of caching and invalidation after first building a real API.
- Keep Zustand focused on client/application state rather than duplicating server data.

Exit criteria

- Remote state is fetched and mutated through TanStack Query.
- The UI handles loading/error/refetch states intentionally.
- Mutations refresh only the data that needs refreshing.
- Network behaviour is covered by appropriate tests.

Workbench 0.6 | Authentication and cloud persistence

Turn Workbench from a single-user local application into a real hosted service with user-owned data.

Scope

- Authentication
- User identity and sessions
- Authorization rules
- Per-user projects/tasks

- Cloud-hosted database
- Secrets/configuration management
- Production-safe error handling

Development principles

- Authentication and authorization are separate concerns.
- Every data operation should have an explicit ownership model.
- Cloud persistence changes operational concerns as well as code.

Exit criteria

- Users can sign in and see only their own data.
- Data survives redeployments and local-machine loss.
- Unauthorized access is rejected server-side.
- Configuration and secrets are not committed to source control.

Workbench 1.0 | Cloud-deployed portfolio release

Polish the accumulated system into a coherent product that is pleasant to use and easy to explain.

Scope

- Cloud deployment of frontend/backend/database
- Responsive UI
- Accessibility review and fixes
- Robust automated test suite
- Production build and deployment workflow
- README with setup and architecture
- Screenshots and feature overview
- Architecture notes explaining key decisions and evolution

Development principles

- Show judgement, not just library usage.
- Document why each major technology was introduced and what problem it solved.
- Keep the product useful rather than turning it into a showcase of unnecessary complexity.

Exit criteria

- Publicly reachable stable deployment.
- Core workflows work on desktop and mobile-sized layouts.
- Accessibility and keyboard usage are deliberately reviewed.
- Tests and documentation make the project credible to another developer or employer.

Beyond 1.0: broader direction

The roadmap above deliberately stops at a coherent Workbench product. It should not pre-design the full whole-life-planner system. However, Workbench may become its first concrete module, and early decisions should avoid making that evolution unnecessarily difficult.

Possible future model

- Plans may become first-class entities rather than merely folders of tasks.
- Plans may connect to other plans as well as tasks, people, finances, events, notes, documents, goals and decisions.
- The eventual system may therefore resemble a graph of interconnected life entities more than a conventional hierarchical todo list.
- These are exploratory design directions, not committed requirements for the Workbench roadmap.

Guiding rule

Do not make version 0.x pay the complexity cost of version 2.x. Keep the broader vision logged, but let real usage create the architectural pressure that justifies each expansion.